

Mazes & Mages: Geração Procedural aplicada a mapas de Dungeons em jogos

Mateus de Oliveira Lopes¹, Josiel Neumann Kuk¹

¹Departamento de Ciências da Computação
Universidade Estadual do Centro-Oeste (UNICENTRO)
, Caixa Postal 3010 – 85040-167 – Guarapuava – PR – Brasil

lopes3137@gmail.com, josiel@unicentro.br

Abstract.

Resumo.

1. Introdução

A indústria brasileira de jogos digitais tem demonstrado um alto crescimento na última década.

2. Fundamentação Teórica

Essa sessão aborda o referencial teórico, com a intenção de fundamentar e contextualizar a presente pesquisa. Serão abordados a geração procedural e sua taxonomia, os elementos da teoria de grafos utilizados na modelagem do mapa junto a triangulação de Delaunay, a otimização por colônia de formigas, os elementos de *game design* utilizados na elaboração dos mapas e, por fim, o funcionamento da /textitgame engine Godot.

2.1. Geração Procedural

A geração procedural (*Procedural Content Generation* - PCG) refere-se à criação de conteúdo por meio de algoritmos, tendo pouca ou nenhuma interferência humana [?]. Sua

2.1.1. Grafos

2.1.2. Algoritmo de Colônia de Formigas

O Algoritmo de colônia de formigas (*Ant Colony Optimization* - ACO) é um algoritmo de otimização que se

2.2. Game Design

2.3. Godot

3. Trabalhos Corelatos

Devido a sua utilização no desenvolvimento de jogos digitais, a maioria

4. Desenvolvimento

A presente sessão descreve todo o processo de desenvolvimento da aplicação, desde a arquitetura do *framework* de geração até a integração com o protótipo jogável. Ela é dividida em duas subseções: a primeira tem foco na lógica de geração, responsável por produzir a estrutura do mapa de forma independente da camada do jogo; e a segunda trata da prototipagem, na qual essa estrutura é consumida, rasterizada e convertida em um ambiente tridimensional explorável pelo jogador.

Essa separação reflete uma decisão arquitetural deliberada: as classes responsáveis pela geração procedural (`GraphBuilder`, `AntColonyOptimizer` e `DungeonGenerator`) estendem `RefCounted` e não possuem qualquer dependência da árvore de cena, dos nós visuais ou do laço de jogo da *engine*. O *framework* comunica-se com a camada de jogo por meio de um único contrato de dados: um dicionário de resultado contendo as posições das salas, as arestas de Delaunay, as arestas selecionadas como corredores, os vértices de início e fim e as métricas estruturais calculadas. Essa fronteira bem definida permite que a lógica de geração seja testada, executada e avaliada isoladamente, ao mesmo tempo em que pode ser integrada a qualquer cena da *engine* que respeite o formato do dicionário.

4.1. Geração Procedural

O processo de geração procedural é orquestrado pela classe `DungeonGenerator`, que coordena as duas etapas algorítmicas fundamentais descritas na proposta deste trabalho: a construção do grafo estrutural da *dungeon* e a aplicação do algoritmo de Otimização por Colônia de Formigas (ACO) sobre esse grafo. O Algoritmo ?? resume essa orquestração.

[1] inicializar gerador de números aleatórios a partir da *seed* **gerar salas** respeitando a separação mínima vértices **construir grafo de Delaunay** sobre as salas corredores potenciais configurar e executar o **ACO** sobre o grafo $start \leftarrow$ sala aleatória $end \leftarrow$ sala mais distante de $start$ *corredores* $\leftarrow$ melhores arestas encontradas pelo ACO calcular métricas (grau médio, coeficiente de agrupamento, conectividade início–fim) dicionário de resultado

Cada etapa é detalhada nas subseções seguintes. A primeira corresponde à definição dos vértices e das conexões candidatas do grafo; a segunda, à seleção, pelo ACO, do subconjunto de arestas que efetivamente compõem os corredores do mapa final; e a terceira descreve as variáveis de controle que parametrizam todo o processo.

4.1.1. Geração das Salas

A primeira etapa, implementada no método `generate_rooms` da classe `GraphBuilder`, é responsável por posicionar as salas que constituem os vértices do grafo. O posicionamento utiliza uma estratégia de **amostragem por rejeição**: um ponto candidato é sorteado uniformemente dentro dos limites espaciais definidos pelo parâmetro `bounds` e só é aceito caso esteja a uma distância igual ou superior à separação mínima (`min_separation`) de todas as salas já posicionadas. Caso contrário, o ponto é descartado e um novo candidato é sorteado.

[caption=Amostragem por rejeição para o posicionamento das salas., label=lst:rooms] while rooms.size() ;

```

room_count and attempts < max_attempts : attempts++ = 1 var p :=
Vector2(rng.randf_range(bounds.position.x, bounds.position.x +
bounds.size.x), rng.randf_range(bounds.position.y, bounds.position.y +
bounds.size.y)) var ok : bool = true for r in rooms : if r.distance_o(p) <
min_separation : ok = false break if ok : rooms.append(p)

```

Esse procedimento produz uma distribuição espacial próxima de um *blue noise*, na qual as salas ficam razoavelmente espalhadas e nunca excessivamente próximas, evitando sobreposições e aglomerações artificiais. O número de tentativas é limitado a um múltiplo do número de salas desejado, de modo que a geração termina mesmo quando a densidade solicitada não pode ser plenamente satisfeita dentro dos limites disponíveis, garantindo a ausência de laços infinitos.

Posicionadas as salas, é necessário definir quais delas podem ser conectadas por corredores. Para isso, o método `build_delaunay_graph` constrói uma **triangulação de Delaunay** sobre o conjunto de pontos, utilizando o algoritmo incremental de Bowyer–Watson [?]. O algoritmo inicia com um *super-triângulo* suficientemente grande para conter todos os pontos; cada sala é então inserida individualmente, removendo-se os triângulos cujo circuncírculo contém o ponto inserido e retriangulando-se a cavidade resultante. Ao final, descartam-se os triângulos que referenciam os vértices auxiliares do super-triângulo. O teste de pertencimento ao circuncírculo é feito por meio de um determinante, ajustado pela orientação dos vértices para garantir robustez numérica independentemente do sentido (horário ou anti-horário) em que o triângulo foi formado.

A triangulação de Delaunay foi escolhida por gerar conexões planares, sem cruzamentos, ligando preferencialmente salas espacialmente próximas. Essa propriedade resulta em uma topologia natural para mapas, na qual cada sala se conecta a vizinhos imediatos, e fornece um conjunto enxuto e bem-comportado de **corredores potenciais**. As arestas extraídas dos triângulos são deduplicadas e armazenadas tanto em uma lista de arestas quanto em uma estrutura de adjacência. Paralelamente, é construída uma **matriz de distâncias** euclidianas entre todas as salas, que será utilizada posteriormente tanto como peso das arestas quanto como informação heurística pelo ACO. Como tratamento de caso degenerado, quando há menos de três salas — situação em que não existe triângulo possível — as salas são simplesmente conectadas em cadeia.

Ao término desta etapa, tem-se o grafo estrutural completo previsto na proposta: cada vértice representa uma sala e cada aresta ponderada representa um corredor potencial, com peso correspondente à distância espacial entre as salas conectadas. A Figura ?? ilustra um grafo gerado, com as salas representadas como círculos e as arestas de Delaunay como conexões candidatas.

Figura 1. Grafo estrutural da *dungeon*: salas (vértices) conectadas pelas arestas candidatas da triangulação de Delaunay.

4.1.2. Geração dos Corredores

Definido o grafo de corredores potenciais, a segunda etapa seleciona quais dessas conexões efetivamente compõem o mapa final. Essa seleção é realizada pela classe `AntColonyOptimizer`, que implementa o algoritmo de Otimização por Colônia de

Formigas [?]. O ACO foi escolhido por construir soluções de forma probabilística e incremental, guiado simultaneamente por uma trilha de feromônio — que reforça as escolhas historicamente boas — e por uma informação heurística — que privilegia escolhas localmente vantajosas.

Inicialmente, todas as arestas recebem um valor de feromônio igual ao parâmetro τ_0 . A cada iteração, um conjunto de formigas percorre o grafo de forma independente. Cada formiga parte de uma sala aleatória e constrói, de modo incremental, uma estrutura conexa que abrange as salas alcançáveis: a partir do conjunto de salas já visitadas, mantém-se uma fronteira de arestas candidatas (que ligam uma sala visitada a uma não visitada), e a cada passo uma aresta da fronteira é escolhida probabilisticamente. A probabilidade de seleção de uma aresta (i, j) é proporcional a

$$p_{ij} \propto \tau_{ij}^\alpha \cdot \eta_{ij}^\beta, \quad \eta_{ij} = \frac{1}{d_{ij}}, \quad (1)$$

em que τ_{ij} é o feromônio depositado na aresta, η_{ij} é a informação heurística (o inverso da distância euclidiana d_{ij} , favorecendo corredores curtos) e α e β ponderam, respectivamente, a influência do feromônio e da heurística. Esse processo prossegue até que a fronteira se esgote, resultando em uma estrutura que conecta todas as salas do componente alcançado a partir da raiz. Por se tratar de uma estrutura geradora conexa, garante-se que exista um caminho entre quaisquer duas salas abrangidas, incluindo, em particular, a conectividade entre a sala inicial e a final exigida pela proposta.

Uma estrutura puramente conexa, contudo, corresponderia a uma árvore, na qual existe um único caminho entre cada par de salas — um *layout* pouco interessante do ponto de vista de jogabilidade, pois eliminaria rotas alternativas e ciclos. Para introduzir variedade estrutural, após a construção da estrutura conexa cada formiga adiciona um conjunto de **arestas de ciclo**: dentre as conexões de Delaunay ainda não utilizadas, seleciona-se probabilisticamente — pelo mesmo critério da Equação ?? — uma quantidade limitada por um orçamento proporcional ao tamanho da estrutura (`loop_factor` $\times$ número de arestas). Essas arestas criam laços e caminhos alternativos, elevando o coeficiente de agrupamento do *layout*.

Cada solução construída é avaliada por uma função objetiva multicriterial que pondera a distância e o agrupamento. Sendo $\bar{d}$ a distância média das arestas do *layout* e C o seu coeficiente de agrupamento médio, o custo é definido como

$$custo = w_d \bar{d} + w_c (1 - C) \bar{d}, \quad fitness = \frac{1}{custo}, \quad (2)$$

de forma que soluções com corredores mais curtos (menor $\bar{d}$) e com maior agrupamento (maior C) recebem maior *fitness*. Os pesos w_d (`w_distance`) e w_c (`w_clustering`) permitem regular o equilíbrio entre economia de corredores e variedade topológica. A melhor solução é mantida segundo um critério lexicográfico: prioriza-se a **cobertura** (número de salas conectadas) e, em caso de empate, o maior *fitness*, assegurando que o *layout* final abranja o maior número possível de salas.

O comportamento do feromônio segue o esquema clássico do ACO, com duas atualizações. A **atualização local** é aplicada a cada aresta percorrida por uma formiga,

aproximando seu feromônio de τ_0 e promovendo a diversificação das soluções exploradas. A **atualização global**, ao fim de cada iteração, reforça as arestas da melhor solução encontrada com um incremento inversamente proporcional ao seu comprimento total, intensificando a busca em torno das boas soluções. Ambas incorporam a taxa de evaporação ρ , conforme:

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij} + \rho \Delta\tau, \quad \Delta\tau_{global} = \frac{1}{L_{melhor}}. \quad (3)$$

Ao final da execução, o método `get_corridor_edges` retorna o conjunto de arestas da melhor solução, que constitui os corredores definitivos do mapa. De volta ao `DungeonGenerator`, a sala inicial é sorteada e a sala final é definida como a mais distante dela em linha reta, maximizando o percurso entre os extremos. Por fim, calculam-se as métricas estruturais sobre o *layout* resultante — grau médio dos vértices, coeficiente de agrupamento médio e verificação da conectividade início–fim por busca em largura —, as mesmas métricas previstas para a avaliação comparativa. A Figura ?? apresenta um mapa final, com os corredores selecionados destacados sobre as arestas candidatas de Delaunay.

Figura 2. *Layout* final: em destaque, os corredores selecionados pelo ACO; em tom esmaecido, as arestas de Delaunay descartadas. Em verde e vermelho, as salas inicial e final.

4.1.3. Parametrização

Conforme estabelecido na proposta, espera-se que os mapas sejam estruturalmente distintos em decorrência da manipulação de variáveis de controle. O *framework* expõe, portanto, um conjunto abrangente de parâmetros que governam tanto a geometria das salas quanto o comportamento do ACO. Todos podem ser ajustados por meio do método `apply_params`, que recebe um dicionário de configuração, desacoplando os valores da implementação. O Quadro ?? relaciona os principais parâmetros e seus efeitos.

Tabela 1. Parâmetros de controle do *framework* de geração.

Parâmetro	Efeito
<code>room_count</code>	Número de salas (vértices) do grafo.
<code>min_separation</code>	Distância mínima entre salas; controla a densidade espacial.
<code>seed</code>	Semente do gerador aleatório; garante reprodutibilidade.
<code>max_iterations</code>	Número de iterações do ACO.
<code>ant_count</code>	Número de formigas por iteração.
α (alpha)	Peso da trilha de feromônio na escolha das arestas.
β (beta)	Peso da informação heurística (inverso da distância).
ρ (rho)	Taxa de evaporação do feromônio.
τ_0 (tau_0)	Valor inicial de feromônio das arestas.
w_d (w_distance)	Peso da distância média na função objetiva.
w_c (w_clustering)	Peso do agrupamento na função objetiva.
<code>loop_factor</code>	Proporção de arestas de ciclo adicionadas ao <i>layout</i> .

Os parâmetros podem ser fixados manualmente ou **aleatorizados** dentro de faixas configuráveis. Quando a aleatorização está ativa, cada parâmetro é sorteado entre um valor mínimo e máximo definidos previamente, de modo que cada geração produza um mapa com características distintas, ampliando a variedade do conjunto de resultados sem necessidade de ajuste manual. O parâmetro *seed* confere reprodutibilidade: fixada uma semente, todo o processo — posicionamento das salas, triangulação e execução do ACO — torna-se determinístico, o que é essencial tanto para a depuração quanto para a reprodução dos mapas durante a etapa de avaliação. Adicionalmente, os limites espaciais (*bounds*) são ajustados automaticamente em função do número de salas e da separação mínima, assegurando área suficiente para acomodar todas as salas solicitadas.

4.2. Prototipagem

Esta subseção descreve a integração do *framework* de geração a um protótipo de jogo desenvolvido na *engine* Godot, com a linguagem GDScript. Enquanto a subseção anterior tratou da produção da estrutura lógica do mapa, esta aborda como essa estrutura é convertida em um ambiente tridimensional explorável, dotado de movimentação, detecção de colisão e renderização, conforme os objetivos definidos na proposta.

4.2.1. Conceitualização

O protótipo, intitulado *Mazes & Mages*, foi concebido como um jogo de RPG de exploração de masmorras em primeira pessoa, com navegação baseada em grade, inspirado em *dungeon crawlers* clássicos do gênero. Nesse formato, o jogador se desloca casa a casa por um ambiente reticulado, o que se alinha naturalmente à representação em grade produzida pela geração procedural e torna a movimentação intrinsecamente compatível com a estrutura do mapa. O objetivo do jogador é simples e serve principalmente como validação da conectividade do mapa: partir da sala inicial e alcançar a sala final.

A arquitetura da camada de jogo organiza-se em torno de alguns nós-chave. A cena *World* atua como orquestradora, integrando o *framework* de geração ao mundo jogável: ela executa o gerador, converte o resultado em uma grade de células, instancia a geometria tridimensional, posiciona o jogador e atualiza a interface. A unidade básica de construção do ambiente é a classe *Cell*, um nó tridimensional que representa uma única casa do mapa, composta por seis faces (chão, teto e quatro paredes). Um arquivo de constantes globais (*Globals*) define o tamanho lógico da grade no mundo, garantindo coerência entre todos os sistemas que convertem coordenadas de grade em coordenadas de mundo.

Um aspecto relevante de desempenho está no método *update_faces* da classe *Cell*: ao instanciar cada célula, as faces compartilhadas entre células de chão adjacentes são removidas, de forma que apenas as paredes externas — aquelas voltadas para o vazio — sejam efetivamente renderizadas. Isso une visualmente salas e corredores contíguos em espaços contínuos e reduz a quantidade de geometria desenhada. Para fins de inspeção e depuração, foi também implementada uma visualização bidimensional em vista superior (*DungeonView*), que desenha as salas, as arestas de Delaunay e os corredores selecionados, permitindo observar diretamente a estrutura lógica do mapa antes de sua conversão para o espaço tridimensional.

4.2.2. Jogabilidade

A movimentação do jogador, implementada na classe `Player`, segue o paradigma de deslocamento discreto característico dos *dungeon crawlers* em grade. Em vez de movimento contínuo, o jogador avança uma casa por vez e gira em incrementos de noventa graus. Um temporizador regula a cadência da entrada, processando uma ação de cada vez e impedindo que comandos se sobreponham enquanto uma transição está em andamento.

A **detecção de colisão** é realizada por meio de quatro nós `RayCast3D`, orientados para frente, trás, esquerda e direita do jogador. Cada raio sonda a célula adjacente na respectiva direção: o deslocamento só é autorizado quando o raio detecta uma célula de chão à frente, ou seja, quando existe uma casa válida para a qual transitar. Dessa forma, o jogador fica confinado aos corredores e salas efetivamente gerados, sem atravessar paredes ou avançar para o vazio. Tanto o deslocamento quanto as rotações são animados por interpolação (*tween*) suave, acompanhados de uma animação de passo, proporcionando uma transição fluida entre casas, apesar da natureza discreta do movimento.

```
[caption=Trecho do processamento de movimento  
do jogador., label=lst:player] if collisioncheck(raydir) :  
timerprocessor.stop()tweenttranslation(getddirection(raydir))
```

A cena `World` é responsável por verificar, a cada quadro, a **condição de vitória**: quando a casa ocupada pelo jogador coincide com a casa da sala final, a cena de saída é carregada, encerrando a exploração. Para apoiar a navegação, a interface dispõe de um **minimapa** (`MinimapView`), que renderiza em vista superior as casas do mapa, destaca as salas inicial e final e acompanha a posição do jogador em tempo real, atualizando-se sempre que este muda de casa. A interface ainda oferece atalhos de teclado para alternar a exibição do mapa e para regenerar instantaneamente uma nova *dungeon*, o que facilita a inspeção visual de múltiplos mapas em sequência. Um painel textual exibe as métricas e os parâmetros da geração corrente, permitindo correlacionar diretamente a configuração utilizada com o mapa produzido. A Figura ?? apresenta o protótipo em execução.

Figura 3. Protótipo *Mazes & Mages* em execução, com o ambiente tridimensional gerado proceduralmente e o minimapa.

4.2.3. Criação dos Mapas de Base

A avaliação proposta para este trabalho compara os mapas gerados proceduralmente com mapas de referência construídos manualmente. Para que essa comparação seja justa, ambos os conjuntos precisam ser representados sobre a mesma estrutura de grade. Essa unificação é alcançada pela classe `GridRasterizer`, responsável por converter a saída abstrata da geração — posições contínuas das salas e lista de corredores — em uma grade discreta de casas.

A rasterização ocorre em duas operações. Primeiro, o centro de cada sala é mapeado para uma coordenada de grade, e um bloco de casas de chão é escavado ao seu redor, com extensão controlada pelo parâmetro `roomradius`. Em seguida, para cada corredor selecionado pelo ACO, escava-se um caminho em formato de "L" entre as casas das salas conectadas — primeiro no eixo horizontal e depois no vertical —, produzindo passagens ortogonais coerentes com a grade. O resultado é um dicionário que associa cada

coordenada ao tipo de casa (chão, parede ou vazio), acompanhado das posições de início e fim, exatamente o formato consumido pela cena `World` para instanciar a geometria.

Os **mapas de base**, por sua vez, são construídos manualmente pelo autor diretamente no editor da *engine*, posicionando as células (`Cell`) à mão para compor a topologia desejada. A classe `ManualMap` cumpre, para esses mapas, o papel que a `World` desempenha para os procedurais: ela percorre recursivamente a árvore de cena, coleta todas as células posicionadas, deriva suas coordenadas de grade, executa a remoção de faces compartilhadas, posiciona o jogador na casa inicial e configura o minimapa. Como os mapas manuais reutilizam exatamente a mesma infraestrutura de células, jogador, minimapa e condição de vitória dos mapas procedurais, ambos tornam-se diretamente equiparáveis.

Essa equivalência de representação é o que viabiliza a avaliação comparativa prevista na proposta: por estarem expressos sobre a mesma estrutura de grade e percorríveis pelo mesmo conjunto de mecânicas, os mapas procedurais e os mapas de base podem ser medidos sob as mesmas métricas estruturais — grau médio dos vértices, coeficiente de agrupamento e conectividade entre a sala inicial e a final —, fornecendo um parâmetro objetivo para aferir a qualidade dos mapas gerados proceduralmente.

5. Resultados

6. Conclusões e Trabalhos Futuros

Referências